BUG PATCHER 멀티플레이 추진계획서

목차

1. 개요
2. 애플리케이션 기획
3. High-level 디자인
4. Low-level 디자인
5. 멀티스레드와 동기화
6. 역할분담
7. 개발환경
8. 개발일정

1. 개요

윈도우 프로그래밍 수업에서 만든 싱글플레이 게임 BUG PATCHER 를 네트워크 게임 프로그래밍 수업에서 2 인 협동 멀티플레이 게임으로 바꾼다.
PC 2 대로 진행하며, PC 1 은 서버와 클라이언트를 함께, PC 2 는 클라이언트만 실행한다.
서버가 게임 판정 전체를 맡고, 두 클라이언트는 입력을 보내고 서버가 보낸 상태를 그린다.

항목	내용
원작	BUG PATCHER, 윈도우 프로그래밍 20 팀 최종 프로젝트
원작 구조	Win32 API + GDI+, 단일 소스 파일(BugPather.cpp, 약 2,100 줄), 타이머 7 개로 로직 처리
이번 목표	2 인 협동 멀티플레이, TCP + UDP 서버-클라이언트 구조, 서버 권한 판정
유지할 콘텐츠	적 3 종, 무기 5 종, 보스 3 종, 맵 2 종(격자 맵, 기믹 맵)
만드는 사람	김현수, 조형진

2. 애플리케이션 기획

게임 내용은 윈도우 프로그래밍에서 완성했고, 이번 학기에는 네트워크 구조와 2 인 규칙을 더한다.

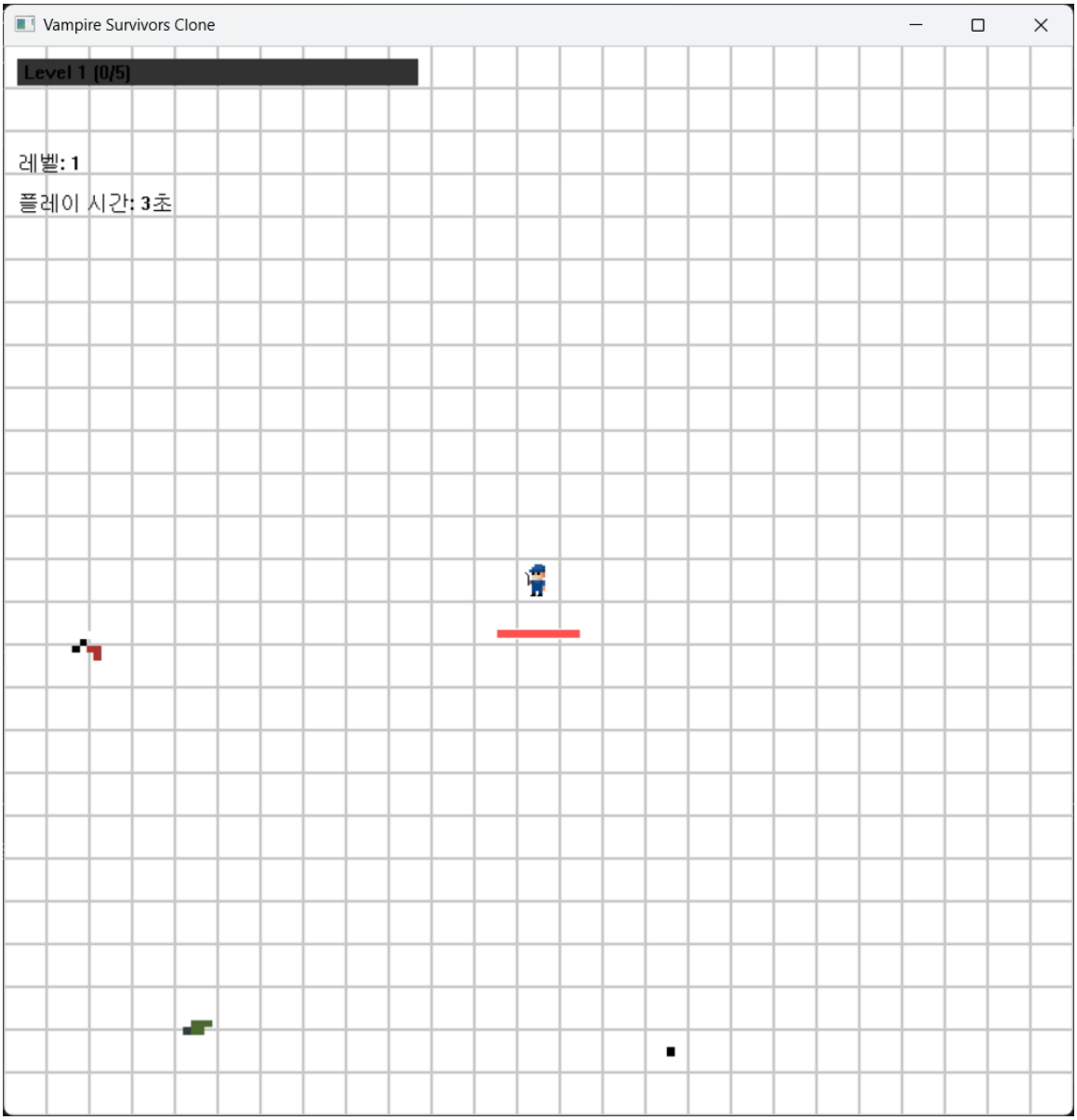
교과목별 작업

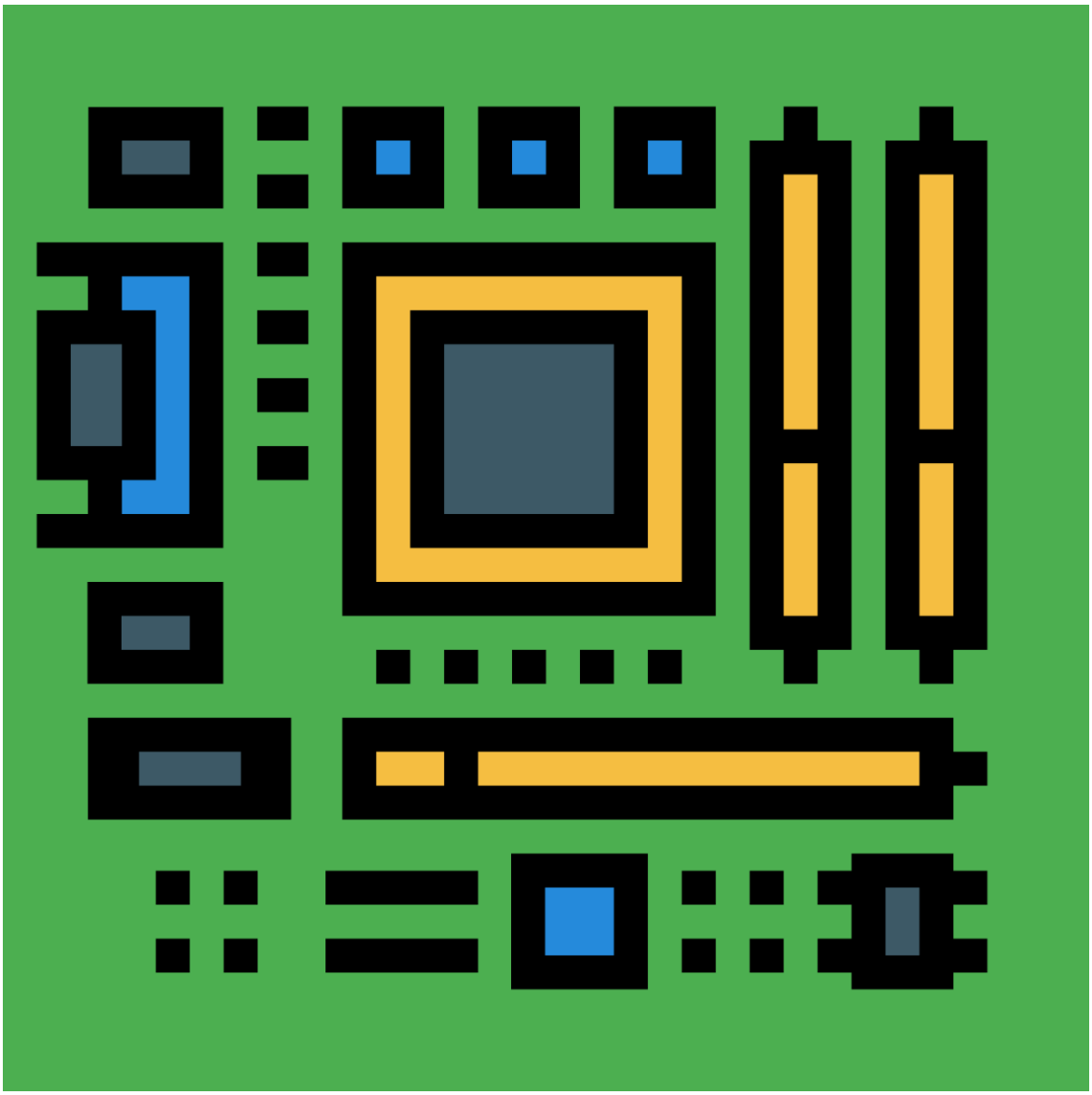
교과목	작업 내용	담당
윈도우 프로그래밍 (20 팀)	게임 기획, Win32/GDI+ 렌더링, 적·보스 AI, 무기 5 종, 기믹 맵, Aseprite 아트	조형진, 홍승우
네트워크 게임 프로그래밍	서버 구축, 프로토콜, 클라이언트 네트워크, 멀티스레드 동기화, 2 인 규칙	김현수, 조형진

스크린샷

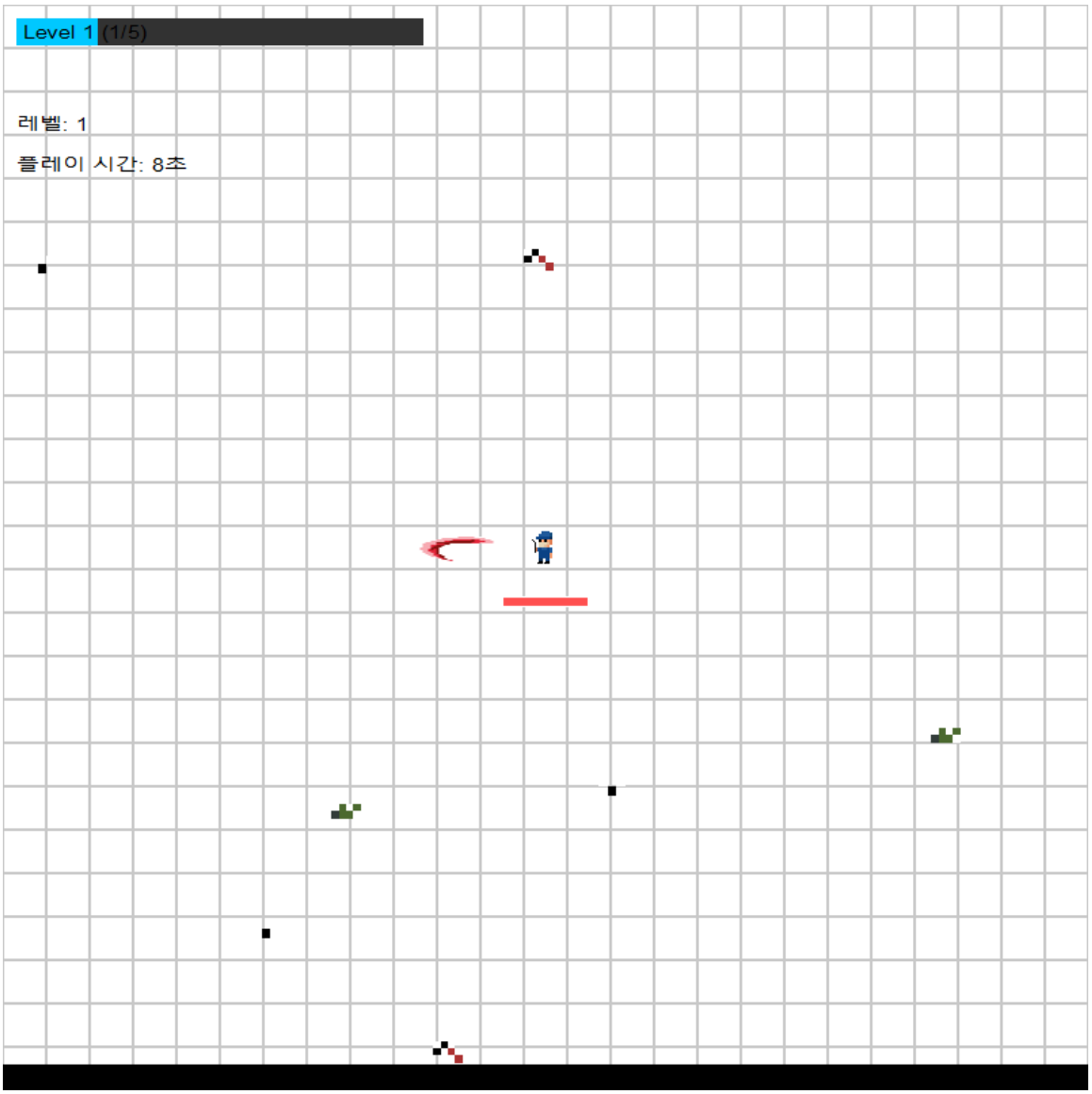


메인 메뉴. Enter 를 누르면 맵 선택으로 넘어간다.





맵 선택 미리보기. 위는 격자 맵(기믹 없음), 아래는 기믹 맵(CD, 바람).



플레이 화면. 파리채가 마우스 방향으로 휘둘러지고, 좌상단에 경험치 바와 플레이 시간이 표시된다.

어떻게 작동하나

1. 메인 메뉴에서 Enter, 맵 선택 화면에서 맵을 클릭한다.
2. WASD 로 이동하고 공격은 자동이다. 파리채와 에프킬라는 마우스 방향을 향한다.
3. 화면 밖에서 1 초마다 벌레가 생기고, 시간이 지날수록 마릿수가 늘어난다(최대 10 마리).
4. 레벨 × 5 마리를 잡으면 레벨업하고, 무기 3 개 중 하나를 골라 얻거나 강화한다(최대 15 레벨).

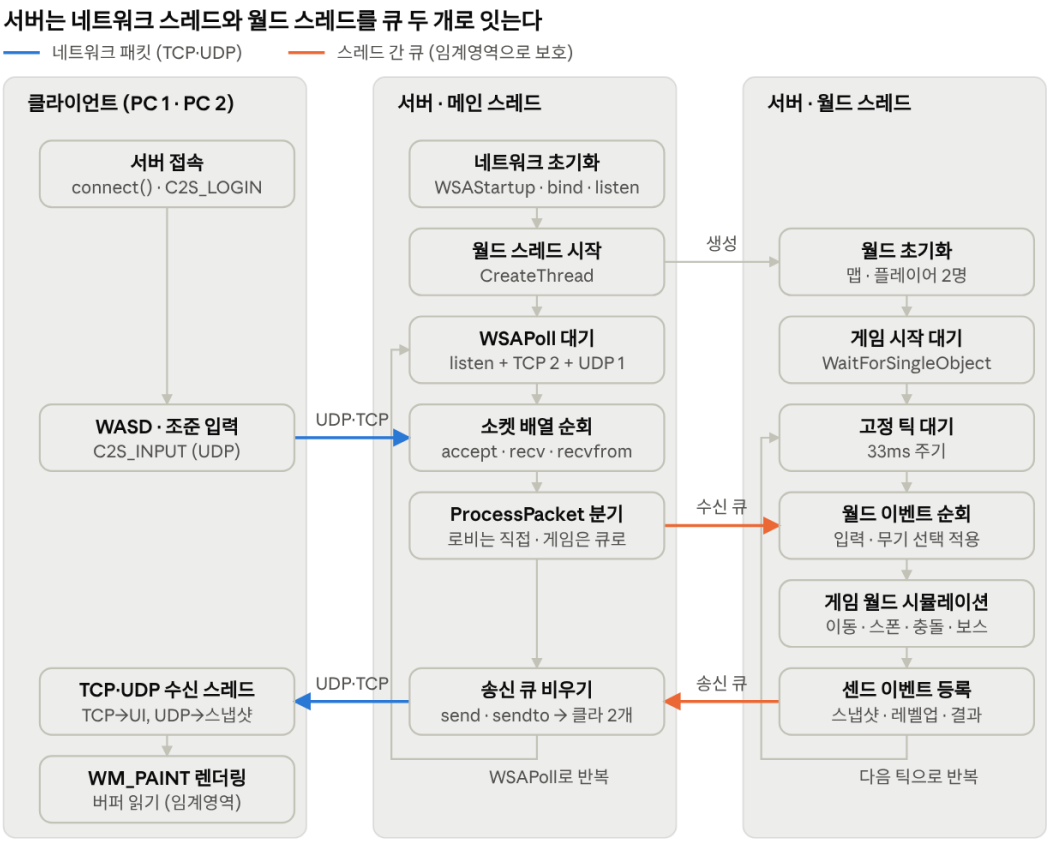
5. 2 분, 3 분, 4 분에 보스 1·2·3 이 나온다. 보스 3 을 잡으면 PATCHED 엔딩, HP 가 0 이 되면 게임 오버.

멀티플레이 규칙

항목	규칙
인원	2 명이 같은 맵에서 동시에 플레이한다
맵 선택	먼저 접속한 1P 가 고른다
적·보스	더 가까운 플레이어를 쫓고 조준한다
레벨·무기	플레이어별로 따로 관리하고, 킬은 마지막으로 때린 플레이어에게 준다(서버가 적마다 마지막 공격자를 기록)
레벨업	고르는 동안에도 월드는 멈추지 않는다. 고르는 중에 다시 레벨업하면 쌓아 두었다가 차례로 고른다
보스 경고	서버는 멈추지 않고, 경고는 클라 화면에만 표시한다
HP 회복	레벨업 때는 본인만, 보스 처치 때는 살아 있는 플레이어 모두 회복한다
적 생성	두 플레이어 화면 밖에서 번갈아 생성하고, 상대 화면 안에 걸리면 위치를 다시 고른다. 생성 수는 살아 있는 인원을 기준으로 밸런스 조정 때 정한다
사망	한 명이 죽으면 관전(부활 없음), 둘 다 죽으면 게임 오버
접속 끊김	나간 플레이어는 사망으로 처리하고, 남은 1 명이 계속 진행한다
클리어	보스 3 을 잡으면 두 명 모두 PATCHED 엔딩

3. High-level 디자인

서버는 메인 스레드가 네트워크를, 월드 스레드가 게임 시뮬레이션을 맡고, 두 스레드는 수신 큐와 송신 큐로만 데이터를 주고받는다. 클라이언트는 입력만 보내고 서버가 보낸 스냅샷을 그린다.



프로그램 흐름 · 클라이언트, 서버 메인 스레드, 서버 월드 스레드

데이터는 한 방향으로 돈다. 클라이언트 입력 → 메인 스레드 recv → 수신 큐 → 월드 시뮬레이션 → 송신 큐 → 메인 스레드 send → 클라이언트 수신 스레드(TCP·UDP) → 화면 순서다. 메인 스레드는 2 명이 접속해 UDP 등록까지 마치고 맵이 정해지면 SetEvent 로 월드 스레드의 '게임 시작 대기'를 풀어 준다.

스레드	루프 한 번에 하는 일
서버 메인	네트워크 초기화, 월드 스레드 시작 후 반복: WSAPoll → 소켓 배열 순회(accept, recv, recvfrom) → 로비 패킷(LOGIN, UDP_HELLO, MAP_SELECT)은 직접 처리, 게임 패킷은 수신 큐 등록 → 송신 큐의 패킷 send/sendto
서버 월드	월드 초기화, 게임 시작 대기 후 반복: 33ms 틱 → 월드 이벤트(수신 큐) 순회 → 게임 월드 시뮬레이션 → 센드 이벤트 등록
클라 메인	메시지 루프: 키·마우스 입력을 C2S_INPUT 으로 sendto(UDP), WM_PAINT 에서

스레드	루프 한 번에 하는 일
	스냅샷 렌더링 (입력은 SetTimer 33ms 마다 보냄)
클라 수신	TCP 수신 스레드: recv → 패킷 조립 → UI 이벤트를 PostMessage / UDP 수신 스레드: recvfrom → SnapshotBuffer 갱신 → InvalidateRect

실행 구성

PC 2 대로 진행한다. PC 1 이 서버와 1P 클라이언트를 함께 실행하고, PC 2 는 2P 클라이언트만 실행한다.

PC	실행 프로그램	서버 접속 주소
PC 1 (호스트)	Server.exe, Client.exe (1P)	127.0.0.1
PC 2	Client.exe (2P)	PC 1 의 LAN IP

- 같은 PC 에서도 서버와 클라이언트는 별도 프로세스다. 클라이언트의 구조와 프로토콜은 PC 1 과 PC 2 가 같고, 접속 주소만 다르다.
- 서버의 TCP·UDP 포트는 고정하고, 클라이언트 UDP 소켓은 따로 bind 하지 않고 첫 sendto 때 OS 가 포트를 정하게 한다. 그래서 PC 1 에서 서버와 클라이언트의 포트가 겹치지 않는다.
- 클라이언트 접속 화면에서 서버 IP 를 입력받는다(기본값 127.0.0.1).
- PC 1 에서 서버를 먼저 실행한 뒤 클라이언트를 켠다. PC 1 의 Windows 방화벽에서 서버 포트(TCP, UDP)를 허용해야 PC 2 가 접속할 수 있다.
- PC 1 의 클라이언트는 루프백이라 지연이 거의 없고 PC 2 는 LAN 지연이 더해진다. 판정은 모두 서버에서 하므로 같은 LAN 에서는 두 플레이어의 차이가 작다.

범위와 위험 요소

- 서버 한 번 실행에 한 판을 진행한다. 결과를 보낸 뒤 서버와 클라를 종료하고, 다시 하려면 서버부터 다시 실행한다.
- 접속이 끊긴 플레이어의 재접속은 지원하지 않는다. 서버(PC 1)가 꺼지면 두 클라 모두 종료 화면으로 간다.

- 클라는 서버가 보낸 스냅샷(약 30Hz)을 그리므로 원작(약 60fps)보다 움직임이 덜 부드러울 수 있고, 클라 예측이 없어 PC 2 는 입력이 조금 늦게 반영된다. LAN 에서 확인한 뒤 필요하면 스냅샷 사이를 보간한다.

4. Low-level 디자인

원작의 전역 변수와 타이머 로직을 서버의 World 클래스로 옮기고, 클라이언트에는 입력과 그리기만 남긴다. 두 쪽이 같이 쓰는 구조체는 Common 폴더에 둔다.

클래스 설계

위치	클래스	역할	담당
Common	Protocol.h	패킷 타입 enum, 패킷 헤더와 패킷 구조체	김현수
Common	GameTypes.h	PlayerState, EnemyState 등 동기화용 구조체와 게임 상수(맵 크기, 무기 수치)	조형진
Server	Server	TCP 리슨 소켓, WSAPoll 루프(UDP 소켓 포함), 세션 배열 관리, 송신 큐 비우기	김현수
Server	Session	클라이언트 TCP 소켓 1 개, playerId, token, 수신 버퍼(패킷 조립), 미전송 송신 버퍼	김현수
Server	UdpEndpoint	UDP 소켓 1 개, recvfrom 처리(UDP_HELLO 주소 등록, C2S_INPUT seq 검사), 등록된 주소로 sendto. 메인 스레드의 WSAPoll 루프가 호출	조형진
Server	World	월드 스레드 본체. Update(dt)에서 원작 TimerProc 로직 실행	조형진
Server	Player, Enemy, Boss, BossBullet, CDGimmick, FanGimmick	원작 구조체를 매개변수화(플레이어 2 명, 적 id)	조형진
Server	EventQueue<T>	CRITICAL_SECTION 으로 보호하는 스레드 간 큐 (수신 이벤트, 송신 이벤트)	김현수
Client	NetworkClient	TCP connect, TCP 수신 스레드, 패킷 조립, TCP	김현수

위치	클래스	역할	담당
		Send 함수	
Client	UdpClient	UDP 소켓, UDP 수신 스레드, SendUdpHello 반복, SendInput(seq)	조형진
Client	SnapshotBuffer	마지막으로 받은 월드 상태. UDP 수신 스레드가 쓰고 메인 스레드가 읽음	조형진
Client	GameClient	원작 WndProc 기반. 입력 수집, 카메라, GifSprite 렌더링, UI	김현수

클라이언트와 서버가 같이 쓰는 데이터

구조체	필드	비고
PlayerState	id, x, y, hp, level, killCount, aimAngle, 무기별 레벨, 무기별 active, 소금총 탄(active, x, y)	무기 이펙트를 그리려면 active 와 각도가 필요. 소금총은 탄이 플레이어와 따로 날아가므로 탄 위치가 있어야 그릴 수 있다
EnemyState	id, type, x, y	id 는 서버가 부여(현재 코드에 이미 있음). hp 는 화면에 쓰지 않아 뺀다
BossState	type, x, y, hp, maxHp, 광역 공격 단계	보스 3 공격 범위 표시에 단계 필요
BulletState	x, y	보스 탄막
GimmickState	종류(CD/바람), x, y, 방향	기믹 맵에서만 사용

클라이언트에만 남는 것: GifSprite, 카메라, 마우스 화면 좌표, UI 상태

서버에만 남는 것: 무기 쿨타임, 보스 패턴 타이머, 스폰 타이머

상태 구조체 (GameTypes.h)

```
// 스냅샷에 담는 상태 (1 바이트 정렬, 필드 타입은 구현하면서 정함)
PlayerState  = { id, 위치, hp, level, killCount, 조준 각도,
                 무기별 레벨[5], 무기별 이펙트 표시, 소금총 탄(active, 위치) }
EnemyState   = { id, type, 위치 }
```

```
BossState    = { type, 위치, hp, maxHp, 광역 공격 단계 }
BulletState  = { 위치 }
GimmickState = { 종류(CD/바람), 위치, 방향 }
```

원작 로직 이식 규칙

- 원작 타이머 7 개(16ms 와 100ms 가 섞여 있음)를 월드 스레드의 33ms 고정 틱 하나로 합친다.
- 원작 수치는 '틱마다 얼마'로 되어 있어 틱 간격이 바뀌면 속도가 달라진다. 이동량은 초당 값에 dt 를 곱하는 방식으로 바꾼다. 예: 플레이어 이동 4px/16ms → 250px/s.
- 접촉 데미지·전기장처럼 100ms 마다 판정하던 것은 마지막 판정 시각을 따로 저장해 100ms 가 지났을 때만 판정한다. 33ms 마다 판정하면 데미지가 약 3 배가 된다.
- 홈매트 둔화 유지 시간(+32ms)처럼 원작 틱 간격에 맞춰 적힌 값은 새 틱 기준으로 다시 정한다.

World 구조

원작의 전역 변수와 하드코딩된 수치를 World 클래스 하나와 수치 표로 정리한다. 싱글 버전을 먼저 고치지 않고, World 로 옮기는 이식 작업을 이 구조로 한다.

원작	문제	World 구조에서는
boss, boss2, boss3 가 각각 전역이고 패턴 변수도 따로(isDashing, isDashing3 등)	보스마다 코드가 복사되어 있고 스냅샷에 따로 담아야 한다	Boss 배열 하나, 패턴 상태는 Boss 안에
playerPos, playerHP, weapons, killCount 가 전역 하나씩	플레이어 2 명을 담을 수 없다	Player players[2]
무기마다 SetTimer(12~15)와 isXxxEnabled 플래그	서버에는 창도 타이머도 없다	WeaponState 의 level·active, Update 에서 처리
로직에서 GetTickCount() 직접 호출	고정 틱과 맞지 않고 테스트가 어렵다	World::nowMs (틱마다 누적)
cameraX, mousePos(화면 좌표)를 로직에서 사용	서버에는 카메라가 없다	C2S_INPUT 으로 받은 keys·aimAngle, 플레이어

원작	문제	World 구조에서는
		위치 기준 시야
무기 수치가 WM_CREATE, 레벨업 처리, 각 타이머에 흩어짐	밸런스 조정 때 여러 곳을 고쳐야 한다	WeaponDef 표 한 곳

```
WeaponDef 표[5] = { 쿨타임, 지속 시간, 기본 데미지, 범위 }        // 수치는 한 곳에

Player = { id, 위치, hp, level, kills,
           입력(keys, 조준 각도, 마지막 seq),
           무기 상태[5](level, active, 마지막 발동 시각), 소금총 탄,
           생존 여부, 남은 레벨업 수 }
Boss   = { type, 위치, hp, maxHp, 패턴 상태(돌진, 방향, 마지막 발사 시각, 광역 단계) }

World
  상태: nowMs, players[2], enemies, bosses, bullets, 기믹들

  Update(dt):
      nowMs += dt
      플레이어 이동 → 무기 처리 → 적 생성·이동 → 보스 패턴 → 기믹
      → 충돌·피해 → 사망·레벨업·종료 판정

  OnInput(playerId, 입력)       : seq 가 최신이면 keys·조준 각도 갱신
  OnSelectWeapon(playerId, 카드) : 무기 획득·강화, 남은 레벨업이 있으면 다음 카드
  OnDisconnect(playerId)         : 사망 처리
  FillSnapshot()                 : Player 등을 PlayerState 등으로 변환
  NearestPlayer(위치)            : 적·보스가 쫓을 대상
```

- 상태는 World 안에만 둔다. 전역 변수를 없애면 플레이어 2 명과 스냅샷 만들기가 자연스럽다.
- 시간은 World::nowMs 를 사용한다. 틱마다 누적하므로 33ms 고정 틱과 맞고, 같은 입력이면 같은 결과가 나와 서버 단독 테스트가 쉬워진다.
- 무기·적·보스 수치는 표 한 곳에 모은다. 2 인 밸런스 조정은 표만 고치면 된다.
- 네트워크로 보내는 상태는 가상 함수 없는 POD 구조체(PlayerState 등)로 두고, FillSnapshot 에서 변환한다. Entity 기반 클래스에 virtual Update 를 두는 상속 구조는 쓰지 않고, 보스 3 종처럼 다른 부분은 type 에 따른 switch 나 패턴 함수로 나눈다.
- 클라이언트의 GifSprite 와 그리기 코드는 원작을 거의 그대로 쓰고, 그릴 때 읽던 전역(playerPos 등)만 스냅샷 값으로 바꾼다.

프로토콜 설계

TCP 와 UDP 를 함께 쓰고, 모든 패킷은 1 바이트 정렬(#pragma pack(1)) 헤더 `{ uint16 size; uint8 type; }` 뒤에 본문이 붙는다. TCP 는 스트림이라 수신 버퍼에 쌓아 두고 size 만큼 모이면 패킷 1 개를 처리하고, UDP 는 데이터그램 하나가 패킷 하나다. 패킷 본문은 양쪽 모두 x86 Windows 라 바이트 순서를 변환하지 않고, 소켓 주소의 IP·포트는 바이트 정렬 함수(htons, inet_pton 등)로 변환한다. 헤더의 size 가 헤더 크기보다 작거나 수신 버퍼보다 크면 잘못된 패킷으로 보고 연결을 끊는다.

패킷	방향	전송	내용	보내는 시점
C2S_LOGIN	클라 → 서버	TCP	닉네임	접속 직후
S2C_LOGIN_OK	서버 → 클라	TCP	playerId(0/1), UDP 등록용 token	로그인 수락
C2S_UDP_HELLO	클라 → 서버	UDP	playerId, token	LOGIN_OK 를 받은 뒤 S2C_UDP_OK 가 올 때까지 반복
S2C_UDP_OK	서버 → 클라	TCP	없음	서버가 그 클라의 UDP 주소를 등록했을 때
C2S_MAP_SELECT	클라 → 서버	TCP	mapId	1P 가 맵을 고를 때
S2C_GAME_START	서버 → 두 클라	TCP	mapId, 두 플레이어 시작 위치	2 명 접속 + UDP 등록 + 맵 선택 완료
C2S_INPUT	클라 → 서버	UDP	seq, WASD 비트 전체, 조준 각도	33ms 마다
S2C_SNAPSHOT	서버 → 두 클라	UDP	tick, 조각 번호/개수, PlayerState×2, 보스·탄막·기믹, 적 배열 조각	매 월드 틱
C2S_SELECT_WEAPON	클라 → 서버	TCP	선택한 카드 번호(0~2)	레벨업 카드 클릭

패킷	방향	전송	내용	보내는 시점
S2C_LEVEL_UP	서버 → 해당 클라	TCP	무기 후보 3 개	킬 수가 기준을 넘을 때
S2C_BOSS_ALERT	서버 → 두 클라	TCP	보스 종류	보스 등장
S2C_GAME_RESULT	서버 → 두 클라	TCP	클리어 / 게임 오버	게임 종료

전송 방식: 매 틱 덮어쓰는 데이터는 UDP, 한 번만 일어나고 꼭 도착해야 하는 패킷은 TCP 로 보낸다. UDP 는 빠지거나 순서가 바뀔 수 있어 다음 규칙을 둔다.

- 순서: C2S_INPUT 에는 seq, S2C_SNAPSHOT 에는 tick 을 넣고, 받은 값이 마지막으로 처리한 값보다 작으면 버린다.
- 입력은 상태 전체: 키를 누름/뗌 이벤트가 아니라 WASD 비트 전체와 조준 각도를 매번 보낸다. 하나가 빠져도 다음 패킷이 덮어쓴다.
- 크기: UDP 패킷 하나는 1,400 바이트 이하로 맞춘다. 넘으면 적 배열을 조각으로 나누고, 각 조각에 tick 과 그 조각에 든 적 목록을 담아 조각 하나만 와도 바로 반영한다(같은 tick 조각을 다 모을 때까지 기다리지 않는다). 조각 번호별로 이미 반영한 tick 보다 오래된 조각은 버린다.
- 조각 수 감소: 새 tick 의 조각이 오면 fragCount 이상 번호로 저장된 조각은 지운다. 적이 줄어 조각 수가 줄면 그 번호의 조각은 다시 오지 않아, 지우지 않으면 옛 조각의 적이 계속 그려진다. 조각 경계는 매 tick 바뀌므로 한 틱 정도 같은 적이 두 조각에 겹쳐 보일 수 있다.
- 조각 나누는 기준: 적 마릿수로 고정하지 않고 남은 바이트로 계산한다. 조각 하나에는 머리 9 + 플레이어 50 + 적 수 2 + 적 7 바이트씩이라 적이 약 190 마리 들어가고, 0 번 조각은 보스·탄막·기믹만큼 줄어든다.
- 주소 등록: 서버는 UDP 소켓 하나로 두 플레이어를 받는다. LOGIN_OK 로 받은 token 을 C2S_UDP_HELLO 에 넣어 보내면, 서버(UdpEndpoint)가 recvfrom 주소를 그 플레이어의 주소로 기록하고 TCP 로 S2C_UDP_OK 를 돌려준다. 등록되지 않은 주소에서 온 UDP 패킷은 버린다.

패킷 구조체 (Protocol.h)

위 패킷 표를 그대로 옮긴 정의다. 모든 패킷은 PacketHeader 로 시작하고, size 에는 헤더를 포함한 전체 바이트 수를 넣는다.

```
PacketHeader = { size(헤더 포함 전체 바이트), type }

C2S_Login        = 헤더 + 닉네임
S2C_LoginOk      = 헤더 + playerId + token
C2S_UdpHello     = 헤더 + playerId + token
S2C_UdpOk        = 헤더
C2S_MapSelect    = 헤더 + mapId
S2C_GameStart    = 헤더 + mapId + 두 플레이어 시작 위치
C2S_Input        = 헤더 + seq + WASD 비트 + 조준 각도
C2S_SelectWeapon = 헤더 + 카드 번호(0~2)
S2C_LevelUp      = 헤더 + 무기 후보 3 개
S2C_BossAlert    = 헤더 + 보스 종류
S2C_GameResult   = 헤더 + 결과(클리어 / 게임 오버)
S2C_Snapshot     = 헤더 + tick + 조각 번호·개수 + 본문(아래 순서)
```

S2C_SNAPSHOT 본문 순서(길이가 바뀌는 패킷이라 순서를 고정한다):

- 모든 조각: S2C_SnapshotHead → PlayerState × 2 → uint16 적 수 → EnemyState 배열. 플레이어는 조각마다 넣어 어느 조각이 와도 내 캐릭터가 갱신된다.
- 0 번 조각에만: 플레이어 뒤에 uint8 보스 수 + BossState 배열 → uint16 탄막 수 + BulletState 배열 → uint8 기믹 수 + GimmickState 배열을 적 배열 앞에 넣는다.

패킷 처리 함수

위치	함수	하는 일
서버 메인	ProcessPacket(Session&, PacketHeader*)	type 으로 분기 후 로비 패킷은 아래 함수로 바로 처리하고, 게임 패킷만 수신 이벤트를 큐에 넣음 (UDP 패킷은 UdpEndpoint 가 받아 보낸 주소로 플레이어를 찾음)
서버 메인	OnLogin, OnUdpHello, OnMapSelect, SendLoginOk, SendUdpOk	로비 단계 패킷을 메인 스레드에서 바로 처리하고 응답을 직접 send 한다. 2 명 로그인·UDP 등록·맵 선택이 끝나면 게임 시작 정보(mapId, 플레이어 2 명)를

위치	함수	하는 일
		기록하고 SetEvent 로 월드 스레드를 깨운다
서버 월드	OnInput, OnSelectWeapon, OnDisconnect	수신 큐에서 꺼낸 이벤트를 월드에 반영 (OnDisconnect 는 나간 플레이어를 사망 처리)
서버 월드	BroadcastGameStart, BroadcastSnapshot, SendLevelUp, BroadcastBossAlert, BroadcastGameResult	송신 이벤트를 만들어 송신 큐에 넣음 (GAME_START 는 깨어난 뒤 시작 위치를 정해 보냄)
클라 수신 스레드	TCP: OnLoginOk, OnUdpOk, OnGameStart, OnLevelUp, OnBossAlert, OnGameResult / UDP: OnSnapshot	받은 값을 SnapshotBuffer 와 UI 상태에 반영
클라 메인	SendLogin, SendMapSelect, SendSelectWeapon(TCP), SendInput(UDP). SendUdpHello 는 UdpClient 가 반복 전송	입력을 패킷으로 만들어 send

큐 이벤트와 경계 함수

스레드 사이에 오가는 이벤트와, 김현수·조형진 코드가 만나는 함수의 모양이다. 나머지 함수는 구현하면서 정한다.

```
// 스레드 간 이벤트
RecvEvent = { 종류(패킷 | 접속 끊김), playerId, 패킷 바이트 복사본 }   // 메인 → 월드
SendEvent = { 대상(P0 | P1 | 전체), UDP 여부, 패킷 바이트 복사본 }    // 월드 → 메인

EventQueue                                          // 김현수
  Push(e)      : 잠금 → 추가 → 해제
  SwapOut(out) : 잠금 → 내부 목록과 out 을 맞바꿈 → 해제

// 서버 메인 (김현수)
Session.OnRecv(데이터) : 수신 버퍼에 붙이고 완성된 패킷마다 ProcessPacket,
                         size 가 이상하면 연결 끊기
ProcessPacket(패킷)    : 로비 패킷은 직접 처리, 게임 패킷은 RecvEvent 로 Push
FlushSendQueue()       : SwapOut 후 TCP 는 send, UDP 는 UdpEndpoint.SendTo
```

```
// 서버 UDP (조형진) – 메인 루프가 호출
UdpEndpoint.OnReadable()        : recvfrom 반복. UDP_HELLO 는 주소 등록, INPUT 은
RecvEvent 로 Push
UdpEndpoint.SendTo(playerId, 패킷)
UdpEndpoint.IsRegistered(playerId) : UDP_OK 전송과 게임 시작 판단에 사용

// 서버 월드 (조형진)
World.ThreadMain():
    게임 시작 Event 대기
    반복: 틱 대기 → 수신 큐 SwapOut → OnInput / OnSelectWeapon / OnDisconnect
          → Update → 스냅샷 등 SendEvent Push

// 클라 UDP (조형진)
UdpClient.Start(서버 IP, playerId, token) : UDP_HELLO 반복 + UDP 수신 스레드 시작
UdpClient.SendInput(keys, 조준 각도)       : seq 를 붙여 sendto
SnapshotBuffer.WriteFragment(조각)        : UDP 수신 스레드가 잠금 안에서 씀
SnapshotBuffer.CopyTo(out)                : WM_PAINT(김현수)가 잠금 안에서 복사만 함
```

- RecvEvent·SendEvent 는 패킷 바이트를 복사해 담는다. 메인 스레드의 수신 버퍼는 다음 recv 에서 다시 쓰이므로 포인터만 넘기면 월드가 읽기 전에 내용이 바뀔 수 있다.
- 세션이 끊기면 메인은 kind = Disconnect 인 RecvEvent 를 넣고, 월드는 OnDisconnect 에서 그 플레이어를 사망으로 처리한다(2 장 접속 끊김 규칙).
- 메인은 SendEvent 의 udp 값으로 소켓을 고른다. true 면 UdpEndpoint 로 등록된 UDP 주소에 sendto, false 면 그 Session 의 TCP 송신 버퍼에 붙여 send 한다.
- World::ThreadMain 은 CreateThread 에 넘기는 정적 함수이고, self 로 받은 World 포인터의 루프를 돈다.

네트워크 I/O 모델

서버는 WSAPoll, 클라이언트는 TCP 수신 스레드와 UDP 수신 스레드를 하나씩 두고 각각 블로킹 recv/recvfrom 을 한다. WSAPoll 은 Select 모델과 같은 원리로, 한 스레드가 여러 소켓을 처리한다.

WSAPoll: pollfd 배열 하나로 listen 소켓, 클라 TCP 소켓 2 개, UDP 소켓 1 개를 감시한다. 네트워크를 메인 스레드 하나가 맡고 월드 스레드와는 큐 2 개로만 연결된다.

WSAPoll 구현 시 지킬 것:

- 서버의 세션 TCP 소켓과 UDP 소켓은 `ioctlsocket(FIONBIO)`로 논블로킹으로 두고, send 가 다 못 보낸 바이트는 Session 송신 버퍼에 남겨 다음 루프에 이어 보낸다.
- POLLOUT 은 송신 버퍼에 남은 데이터가 있을 때만 켠다. 항상 켜두면 WSAPoll 이 계속 바로 리턴해 CPU 를 다 쓴다.

- POLLHUP·POLLERR 이 오거나 recv 가 0 을 돌려주면 세션을 정리하고, 월드에 Disconnect 이벤트를 넣는다.
- 클라이언트는 TCP 수신 스레드(김현수)와 UDP 수신 스레드(조형진)를 따로 둔다. 서로 다른 소켓만 다루므로 각자 따로 테스트할 수 있다. 송신은 메인 스레드에서 바로 send/sendto 한다.

소켓 옵션: 작은 패킷의 지연을 줄이도록 TCP 소켓에 TCP_NODELAY 를, 서버 재실행 시 bind 오류를 막도록 SO_REUSEADDR 를 적용한다. 버퍼 크기 등 나머지 옵션은 테스트하며 정한다.

스레드 구현

프로세스	스레드	담당	생성	루프
서버	메인(네트워크)	김현수	프로그램 시작	WSAPoll(타임아웃 수 ms) → accept/recv, UDP 소켓은 UdpEndpoint::OnReadable → 로비 패킷 처리 또는 수신 큐 등록 → 송신 큐 비우기(TCP send, UDP 는 UdpEndpoint::SendTo)
서버	월드	조형진	메인이 CreateThread(World::ThreadMain)	게임 시작 이벤트 대기 → 33ms 고정 틱마다 수신 큐 처리 → Update → 송신 큐 등록
클라	메인(UI)	김현수	프로그램 시작	메시지 루프, 33ms 타이머로 UdpClient::SendInput, 스냅샷 렌더링, WM_APP 으로 받은 TCP 이벤트를 UI 에 반영
클라	TCP 수신	김현수	connect 성공 후 CreateThread	블로킹 recv → 패킷 조립 → OnLoginOk 등 처리 → PostMessage(WM_APP)로 메인에 전달
클라	UDP 수신	조형진	LOGIN_OK 후 UdpClient::Start	블로킹 recvfrom → WriteFragment(SnapshotBuffer) →

프로세스	스레드	담당	생성	루프
				InvalidateRect

5. 멀티스레드와 동기화

공유 데이터 보호는 임계영역, 게임 시작 알림은 이벤트를 쓴다.

공유 자원	쓰는 스레드	읽는 스레드	동기화 객체
수신 이벤트 큐 (서버)	메인	월드	CRITICAL_SECTION
송신 이벤트 큐 (서버)	월드	메인	CRITICAL_SECTION
게임 시작 신호 (서버)	메인 (SetEvent)	월드 (WaitForSingleObject)	수동 리셋 Event
게임 시작 정보 (서버)	메인 (SetEvent 전에 한 번 씀)	월드 (깨어난 뒤 읽음)	Event 로 순서 보장, 잠금 없음
SnapshotBuffer (클라)	UDP 수신	메인(WM_PAINT)	CRITICAL_SECTION
UI 이벤트 (클라)	TCP 수신	메인	PostMessage(WM_APP). 메시지 큐가 순서를 보장하므로 잠금 없음

임계영역을 고른 이유

- 공유하는 스레드가 모두 같은 프로세스 안에 있다. 뮤텍스는 프로세스 간 동기화용 커널 객체라 여기서는 필요 없다.
- 임계영역은 경합이 없을 때 커널 모드로 들어가지 않아 빠르다. 큐는 매 틱 여러 번 잠기므로 비용이 중요하다.
- 잠금 안에서는 큐를 로컬 큐와 맞바꾸기(swap)만 하고, 처리는 잠금 밖에서 한다. 잠금 시간이 짧아지고 send/recv 가 잠금을 잡고 멈추는 일이 없다.
- 클라이언트 SnapshotBuffer 도 같은 원칙을 쓴다. WM_PAINT 는 잠금 안에서 스냅샷을 지역 변수로 복사만 하고, 그리기는 잠금 밖에서 한다. 그리는 동안 수신 스레드가 기다리지 않는다.

이벤트를 고른 이유

- 월드 스레드는 2 명이 접속하고 맵이 정해질 때까지 할 일이 없다. 바쁜 대기 대신 WaitForSingleObject 로 잠들었다가 메인의 SetEvent 로 깨어난다.
- 수동 리셋으로 두어, 한 번 시작하면 게임이 끝날 때까지 신호 상태를 유지한다.
- 메인 스레드가 SetEvent 전에 써 둔 게임 시작 정보(mapId, 플레이어)는 WaitForSingleObject 로 깨어난 월드 스레드에서 그대로 보인다. 게임이 시작되면 메인은 로비 패킷을 무시하고 이 값을 다시 쓰지 않으므로 따로 잠그지 않는다.

주의할 점: WSAPoll 은 이벤트 객체를 기다릴 수 없다. 그래서 메인 스레드는 WSAPoll 에 짧은 타임아웃을 주고, 매 반복마다 송신 큐를 확인해 보낸다. 타임아웃 값은 구현 때 측정해 정한다.

6. 역할분담

2 명이 서버 네트워크와 클라이언트를 TCP·클라이언트 화면(김현수)과 서버 World·UDP(조형진)로 나눠 맡는다.

팀원	담당 영역	구현하는 네트워크 기능	그 외
김현수	TCP 통신, 서버 메인 스레드, 클라이언트 화면	서버: WSAStartup·listen·accept, WSAPoll 루프(UDP 소켓도 poll 배열에 넣고 UdpEndpoint 호출), Session 수신 버퍼와 TCP 패킷 조립, 송신 큐 처리(TCP send, UDP 는 UdpEndpoint::SendTo 호출), 로비 패킷(OnLogin, OnMapSelect, SendLoginOk, SendUdpOk), 게임 시작 Event(SetEvent), 접속 끊김 → Disconnect 이벤트, EventQueue(CRITICAL_SECTION) 구현, 월드 스레드 생성. 클라: TCP connect, TCP 수신 스레드와 패킷 조립, TCP 패킷 처리(OnLoginOk, OnUdpOk, OnGameStart, OnLevelUp, OnBossAlert, OnGameResult)를 PostMessage 로 UI 에 전달, SendLogin·SendMapSelect·SendSelectWeapon	Protocol.h, 클라 렌더링(SnapshotBuffer::CopyTo 결과로 그리기), 접속(IP 입력)·로비·레벨업 카드·결과·관전 UI, 방화벽 포트 설정
조형진	서버 World	서버: World::ThreadMain(게임 시작 Event 대기,	GameTypes.h, 원작

팀원	담당 영역	구현하는 네트워크 기능	그 외
	스레드, UDP 통신, 스냅샷	33ms 틱, 수신 큐 SwapOut, 송신 큐 Push), UdpEndpoint(UDP bind, recvfrom, C2S_UDP_HELLO 주소 등록, C2S_INPUT seq 검사, sendto), 게임 패킷 처리(OnInput, OnSelectWeapon, OnDisconnect), 송신 이벤트 생성(BroadcastGameStart, BroadcastSnapshot, SendLevelUp, BroadcastBossAlert, BroadcastGameResult), 스냅샷 직렬화와 UDP 조각 분할. 클라: UdpClient(UDP 소켓, UDP 수신 스레드, SendUdpHello 반복, SendInput 33ms·seq), SnapshotBuffer(WriteFragment 의 tick·fragCount 규칙, CRITICAL_SECTION)	로직 World 이식(dt 변환, WeaponDef 표), 2 인 규칙, 밸런스 조정

7. 개발환경

항목	내용
OS	Windows 10/11 x64
IDE / 컴파일러	Visual Studio 2026, C++17
네트워크	Winsock2 ,TCP + UDP, WSAPoll
멀티스레드	Win32 CreateThread, CRITICAL_SECTION, Event
그래픽	Win32 API + GDI+ 원본 소스
아트	Aseprite
형상 관리	Git, GitHub
테스트	한 PC 에서 루프백(127.0.0.1)으로 클라 2 개 실행, 이후 같은 LAN 의 PC 2 대(PC 1: 서버 + 클라, PC 2: 클라만). 한 PC 에서는 포커스가 있는 창만 키 입력을 받으므로, 디버그용 자동 이동 입력 옵션을 둔다

8. 개발일정

색을 칠한 칸은 단계 확인일과 제출일, 회색 칸은 개발 기간 밖이다. 빈 칸은 작업을 잡지 않은 날이다.

김현수 · TCP 통신, 서버 메인 스레드, 클라이언트 화면

월	화	수	목	금	토	일
11/2	**11/3** Protocol.h, 경계 인터페이스 확정(전원)	**11/4** 솔루션 3 개로 분리(전원)	**11/5** 서버 소켓 초기화, WSAPoll 루프	**11/6** accept, Session, TCP 패킷 조립	**11/7**	**11/8**
11/9 로그인, UDP 등록 응답, EventQueue	**11/10** 클라 TCP 접속·수신 스레드, 접속 화면	**11/11** 맵 선택, 게임 시작 흐름, 월드 스레드 생성	**11/12** 플레이어 화면·카메라, 송신큐 정리	**11/13** **1 단계 확인: 접속과 이동(전원)**	**11/14**	**11/15**
11/16 부분 send	**11/17** 적·HP 화면	**11/18** 접속 끊김 처리	**11/19** 로비·맵 선택 화면	**11/20** **2 단계 확인: 적(전원)**	**11/21**	**11/22**
11/23 무기 이펙트 화면	**11/24** 소금총 탄·경험치 화면	**11/25** 레벨업 카드	**11/26** 통합 테스트(전원)	**11/27** 버그 수정	**11/28** **3 단계 확인: 공격과 레벨업(전원, PC 2 대)**	**11/29**
11/30	**12/1**	**12/2**	**12/3**	**12/4**	**12/5**	**12/6**

월	화	수	목	금	토	일
보스·탄막 화면, 보스 경고	기믹 화면, 방화벽·포트 설정	**4 단계 확인: 보스와 기믹, LAN 테스트(전원, PC 2 대)**	결과·관전 화면	버그 수정, 코드 정리		
12/7 **5 단계 확인: LAN 최종 테스트(전원)**	**12/8** 보고서·발표 준비	**12/9** **제출**	12/10	12/11	12/12	12/13

조형진 · 서버 World 스레드와 UDP 통신

월	화	수	목	금	토	일
11/2	**11/3** GameTypes.h, 경계 인터페이스 확정(전원)	**11/4** 솔루션 3 개로 분리(전원)	**11/5** World 골격, 플레이어 이동 이식	**11/6** 월드 스레드(틱 루프, 큐 처리)	**11/7**	**11/8**
11/9 서버 UDP 수신·주소 등록	**11/10** 클라 UDP 입력 송신·수신 스레드	**11/11** 입력 반영, 플레이어 스냅샷 송신	**11/12** SnapshotBuffer	**11/13** **1 단계 확인: 접속과 이동(전원)**	**11/14**	**11/15**
11/16 적 스폰·이동 이식	**11/17** 가까운 플레이어 추적, 접촉 피해	**11/18** 스냅샷 적 배열·조각 분할	**11/19** 조각 수신 규칙 점검	**11/20** **2 단계 확인: 적(전원)**	**11/21**	**11/22**
11/23 무기 이식(파리채·전기장·에프킬라)	**11/24** 무기 이식(홈매트·소금총), 킬 판정	**11/25** 경험치·레벨업 처리	**11/26** 통합 테스트(전원)	**11/27** 버그 수정	**11/28** **3 단계 확인: 공격과 레벨업(전원, PC 2 대)**	**11/29**
11/30 보스 3 종·탄막 이식, 보스	**12/1** CD·바람 기믹 이식	**12/2** **4 단계 확인: 보스와 기믹, LAN**	**12/3** 사망·관전·결과 판정, 접속 끊김	**12/4** 2 인 밸런스 조정, UDP 손실·순서	**12/5**	**12/6**

월	화	수	목	금	토	일
알림		**테스트(전원, PC 2 대)**	처리	점검		
12/7 **5 단계 확인: LAN 최종 테스트(전원)**	**12/8** 보고서·발표 준비	**12/9** **제출**	12/10	12/11	12/12	12/13